Agentic RL for Code

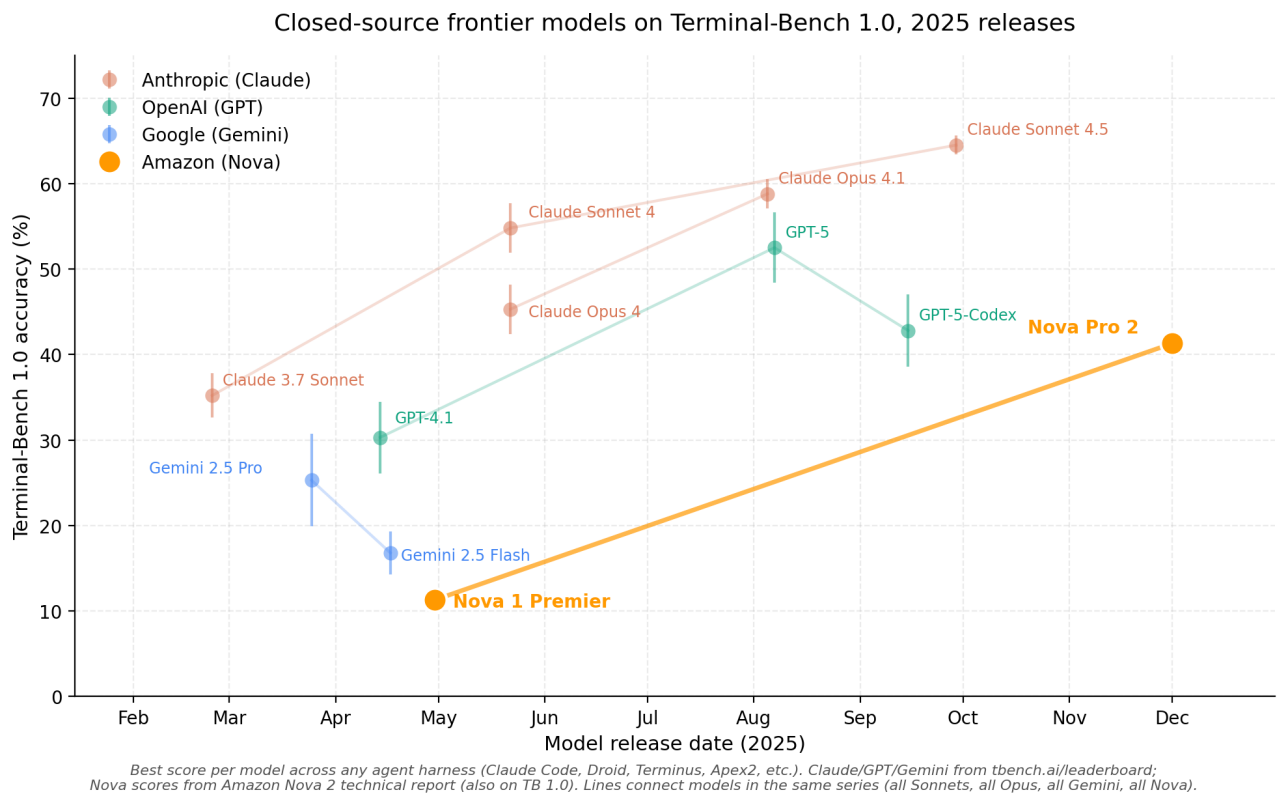
Circa 2025

Anurag Pratik

Amazon

Closed-source frontier on Terminal-Bench

A snapshot of 2025



Training frontier agentic RL models

Four components that determine outcomes

Infra

Async RL (e.g. **PipelineRL**) is key to fast training — concurrent rollouts and in-flight weight updates. Frontier scale still demands bespoke infra.

PipelineRL: ServiceNow Research, arxiv 2509.19128

Data / Environments

Training tasks for generalization, real-world use.

Algorithm

Methods: **GRPO, CiSPO, DAPO.** Tricks to make it work at scale.

Rewards:

- Easy data: terminal-only
- Hard data: path-dependent (scaffolding)

Bitter lesson on easy data; finer-grain rewards break the bootstrapping trap on hard data.

Evals

Must be **decontaminated, representative,** and use appropriate metrics (e.g. rubric-based scoring).

Training frontier agentic RL models

Today's focus: Data / Environments and Algorithm

Infra

Async RL (e.g. PipelineRL) is key to fast training — concurrent rollouts and in-flight weight updates. Frontier scale still demands bespoke infra.

PipelineRL: ServiceNow Research, arxiv 2509.19128

Data / Environments

Training tasks for generalization, real-world use.

Algorithm

Methods: **GRPO, CiSPO, DAPO.** Tricks to make it work at scale.

Rewards:

- Easy data: terminal-only
- Hard data: path-dependent (scaffolding)

Bitter lesson on easy data; finer-grain rewards break the bootstrapping trap on hard data.

Evals

Must be decontaminated, representative, and use appropriate metrics (e.g. rubric-based scoring).

Data: harnesses

Foundation models need to train across many harnesses and datasets to generalize

OpenHands

ICLR '25 · open source

Generalist agent platform

SYSTEM PROMPT

```
You are OpenHands agent...
Combine multiple bash
commands into one, using
sed and grep to edit/view
multiple files at once.
...Use find, grep, and git
commands with appropriate
filters to minimize
unnecessary operations.
```

Cline

VS Code extension

Open-source coding agent

SYSTEM PROMPT

```
Tool use is formatted using
XML-style tags:
  <read_file>
    <path>src/main.js</
path>
  </read_file>
...Use execute_command,
write_to_file,
replace_in_file,
search_files, list_files,
browser_action...
```

Kiro

Amazon · Code OSS

Spec-driven agentic IDE

SYSTEM PROMPT

```
The model MUST create an
implementation plan at
.kiro/specs/{feature}/
tasks.md
...Format as a numbered
checkbox list with a max of
two levels of hierarchy.
...Convert the feature
design
into a series of prompts
for
test-driven implementation.
```

Agentless

Xia et al. '24 · arxiv 2407.01489

Workflow pipeline — no agent tool use

SYSTEM PROMPT

```
### GitHub Problem ###
{issue_description}

### Repository Structure
###
{file_tree}

Please look through the
problem description and
provide a list of files
to edit. Return ≤ 5 files.
```

Datasets covered next: SWE-Gym (ICML '25) · R2E-Gym (COLM '25) · SWE-smith (NeurIPS '25)

Data: how the SWE datasets are built

Real issues → commit-grounded synthesis → injected bugs (scale grows as authenticity drops)

SWE-Gym

2,438 tasks · ICML '25

Real human-reported issues

1. Filter Python repos: 500+ stars, 500+ PRs, 100+ contributors → 358 candidates
2. Extract issue + PR + unit tests (SWE-Bench-style script)
3. Configure executable env per instance (manual)

Most authentic, smallest scale, costly to scale.

R2E-Gym

8,135 tasks · COLM '25

Commit-grounded synthesis (SWE-Gen)

1. Pick high-quality repos (13 total)
2. Mine commits directly — no GitHub issues needed
3. Auto-generate unit tests for the change
4. Back-translate code change to natural-language task

No humans in the loop; bound to real commits.

SWE-smith

50K tasks · NeurIPS '25 Spotlight

Bug injection at scale

1. Pick Python codebases (128 repos)
2. Inject bugs — 4 strategies: LM-Modify, LM-Rewrite, AST mutation, PR-Mirror (undo a real PR)
3. Validate: each candidate must break ≥1 existing test
4. Generate issue text via LM

Most synthetic, largest scale, ~$0.02 per task.

Training pipeline

Where mid-training and RL sit in the broader picture

1

Pretraining

self-supervised

Next-token prediction on web + code corpora.

`~trillions of tokens`

2

Mid-training

continued pretraining

Specialized data: execution traces, agentic Docker rollouts, code-domain corpora.

`~10s of B tokens`

3

Instruction tuning

SFT

Cross-entropy on instruction-following + helpful behavior demonstrations.

`~100K — 1M examples`

4

RL

RLVR / RLHF

Policy gradient on verifiable tasks — terminal or path-dependent rewards.

`~10K — 100K prompts`

Today: *mid-training recipes (next two slides) and RL algorithm choices (later).*

Data: Midtraining

Grounding code agents in execution data before RL — two recent recipes

Code World Model

Meta FAIR · arxiv 2510.02387 · 32B open-weight

Mid-train on execution traces, not just static code

- **Python interpreter traces**
  Local variable state recorded after every line — teaches how state evolves.
- **Agentic Docker trajectories**
  Edits, shell commands, test feedback inside executable repo images.
- **Scale**
  ~3M trajectories across ~10K repo images; collected via "ForagerAgent".

Result: SWE-bench Verified: 53.9% (65.8% w/ TTS)

SWE-Zero → SWE-Hero

NVIDIA · arxiv 2604.01496

Bifurcated SFT: execution-free breadth, then execution-backed refinement

- **SWE-Zero (execution-free)**
  300K trajectories distilled from Qwen3-Coder-480B — masters code semantics & repo-level reasoning without runtime feedback.
- **SWE-Hero (execution-backed)**
  13K trajectories from real Docker environments — grounds semantic intuitions in test-verified workflows.
- **Why it scales**
  Step 1 doesn't need executable envs, so it can mine the long tail of GitHub.

Result: SWE-bench Verified: 62.2% (32B model)

Midtraining: A Case Study for improving Localization

Closing a gap in the agentless harness with synthetic localization tasks

1 Observed
Agentless harness — the localization stage picks the wrong files / functions on many tasks.

2 Built
Synthetic localization tasks added to the midtraining data mix.

3 Improved
Localization accuracy on the agentless harness lifted across internal evals.

Task 1 · Files from issue

Given a PR or issue description, list the files that need to be modified.

INPUT

"RuntimeError when DataLoader is used with num_workers>0 and pin_memory=True — pinned tensors aren't released between epochs, OOM after a few iters."

↓

OUTPUT

```
torch/utils/data/dataloader.py
torch/utils/data/_utils/pin_memory.py
test/test_dataloader.py
```

Task 2 · Function from description

Given a function description, return its name and path.

INPUT

"Applies linear warmup followed by cosine decay to the learning rate; returns a LambdaLR scheduler."

↓

OUTPUT

```
get_cosine_schedule_with_warmup
transformers/optimization.py
```

Algorithm

Loss · Length · Scaling · Rewards · Curriculum

Algorithm: loss functions

GRPO → CiSPO → DAPO: what each objective does

Methods

GRPO

$$\mathcal{L}_{\text{GRPO}} = -\mathbb{E}\left[\min\left(r_t \hat{A}_t,\ \text{clip}(r_t,\ 1-\varepsilon,\ 1+\varepsilon)\,\hat{A}_t\right)\right]$$

group-relative advantage, no value model · DeepSeek-Math (2024)

CiSPO

$$\mathcal{L}_{\text{CiSPO}} = -\mathbb{E}\left[\text{sg}\left(\text{clip}(r_t,\ 1-\varepsilon_{\text{low}},\ 1+\varepsilon_{\text{high}})\right)\cdot \hat{A}_t \cdot \log \pi_\theta(a_t \mid s_t)\right]$$

clip IS weight, keep gradient on all tokens · MiniMax M1 (2025)

DAPO

$$\mathcal{L}_{\text{DAPO}} = -\mathbb{E}\left[\min\left(r_t \hat{A}_t,\ \text{clip}(r_t,\ 1-\varepsilon_{\text{low}},\ 1+\varepsilon_{\text{high}})\,\hat{A}_t\right)\right]$$

asymmetric clip + dynamic sampling · ByteDance Seed (2025)

Notation: *$r_t = \pi\vartheta/\pi_{old}$ · $\hat{A}_t$ = group-normalised reward · sg = stop-gradient (detach)*

✓ CiSPO

Our choice

Gradient flows through **every token** — including the low-probability reasoning tokens ("wait", "however", "aha") that PPO/GRPO/DAPO clipping would drop.

Algorithm: stabilizing length

Reasoning length explodes in long RL runs — RL + SFT replay keeps it bounded

Length explosion: across multi-day RL runs, completion lengths drift upward — the model rewards itself for thinking longer even when the task doesn't need it.

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{CiSPO}}(\theta) + \alpha \cdot \mathcal{L}_{\text{SFT}}^{\text{replay}}(\theta)$$

$$\mathcal{L}_{\text{SFT}}^{\text{replay}}(\theta) = -\mathbb{E}_{(x,y)\sim\mathcal{D}_{\text{replay}}}[\log \pi_\theta(y \mid x)]$$

$\mathcal{L}_{\text{CiSPO}}$

Policy gradient

Standard RL objective from earlier — drives task performance.

$\mathcal{L}_{\text{SFT}}^{\text{replay}}$

Minimal-thinking replay

Cross-entropy on a held-out set of short, concise demonstrations that anchor brevity.

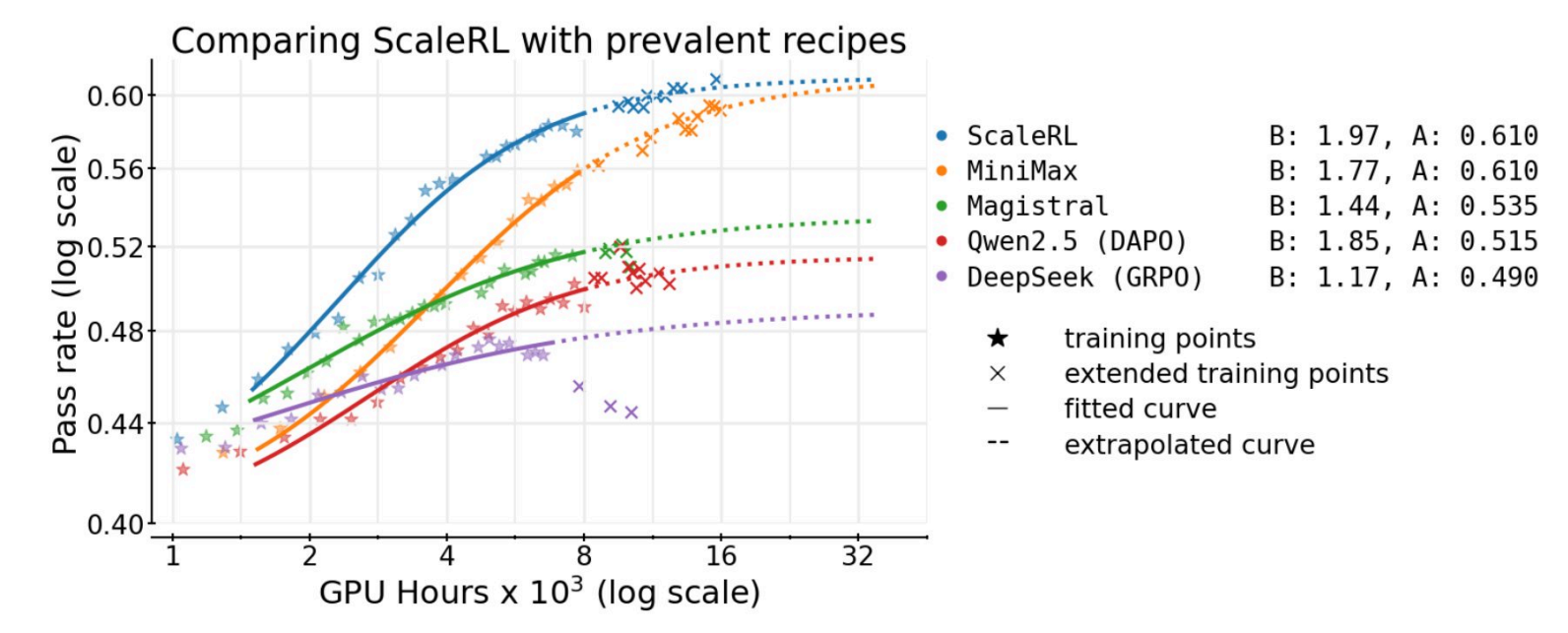
α

Mixing coefficient

Small scalar weighting the SFT pull. Tune to balance task gains against length stability.

Algorithm: scaling

Pass rate vs compute across prevalent RL recipes



Challenges

- Training–inference mismatch
- Router replay (MoE)
- Sequence packing

Algorithm: rewards

Bitter lesson on easy data — scaffolding rewards on hard data

Easy data

most rollouts succeed

Terminal reward suffices

the bitter lesson kicks in

Group-relative advantages give clean gradient when most rollouts in a group achieve nonzero reward — the terminal signal alone drives learning. Hand-engineered reward channels (rubric, process) add value early but fail to compound as compute scales.

→ *Resist adding structure as compute scales.*

Hard data

most rollouts fail

Finer-grain, path-dependent rewards

break the bootstrapping trap for RLVR

When the whole group fails, group-relative advantages collapse to ≈0 — no gradient to bootstrap from. Path-dependent rewards (process traces, partial credit, intermediate checks) inject variance into the group and restore learning signal until terminal reward becomes learnable.

→ *Taper toward terminal-only as pass rate climbs.*

Related: DAPO addresses the same regime via *dynamic sampling* — filter all-zero-reward groups at sample time instead of densifying the reward (ByteDance Seed, arxiv 2503.14476).

Algorithm: curriculum

Profile → filter → shift: a pass-rate curriculum that adapts as the policy improves

Mechanism (Nemotron 3 Nano §3.2.2)

1 Profile
Run SFT checkpoint on every RL task; record pass rate per task.

2 Filter
Drop tasks the SFT model already solves at 100% — no learning signal.

3 Shift
Sample from a Gaussian over pass rate whose mean drops linearly: easy → hard.

Pass-rate distribution over training (schematic)

*When training plateaus, **re-profile** with the best RL checkpoint and build a new curriculum — outperforms random sampling, which biases toward easy tasks.*

Schematic illustrating the mechanism. See Fig 6/7 in Nemotron 3 Nano tech report (arxiv 2512.20848) for empirical curves and curriculum-vs-random comparison.

Future directions

Some directions I find interesting

1

Open weights → real-world deployment

Vertical-specific systems built on open-source models, deployed where they create measurable economic value.

- **Applied Compute**
  "Specific Intelligence" — custom RL agents for enterprises

2

Rewards: weakly verifiable, real-world

Verification expands beyond clean unit tests to noisier, longer-horizon, economically real tasks.

- **TB3**
  "Real compensated computer work" — tasks a human would be paid to do, with robust programmatic verification.

3

Knowledge discovery with LLMs

AlphaEvolve already pushes algorithm and math discovery via verifiable evaluators. Real-world science — chemistry, biology, materials — needs verification in the lab, not in code.

- **Periodic Labs**
  AI scientists + automated labs for materials science — closing the loop on physical-world verification.